

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа № 3
Распределённая модель обработки данных

Студент группы ИКБО-11-23, Дицель Н.С.

(подпись)

Проверил Старший преподаватель,
Приходько Н.А.

(подпись)

Отчет представлен «19» сентября 2026 г.

Москва 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Подготовка окружения 3
1.1 Установка и проверка версий 4
1.2 Выгрузка таблиц SQLite 4
2 Учебная часть: анализ базы ecommerce 6
2.1 Запуск SparkSession 6
2.2 Чтение данных и схема 6
2.3 Преобразования, пропуски и агрегации 8
2.4 Соединения, Spark SQL и запись результата 11
3 Самостоятельная работа 14
3.1 Набор данных 14
3.2 Схема и вычисляемые признаки 14
3.3 Пропуски 16
3.4 Агрегации, соединения и Spark SQL 16
3.5 Экспорт результата 18
ЗАКЛЮЧЕНИЕ 20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 21

ВВЕДЕНИЕ

Apache Spark – вычислительный движок для параллельной обработки больших наборов табличных данных. Он отделяет описание преобразований от их выполнения: пользователь строит цепочку операций над DataFrame, а Spark лениво формирует план и исполняет его по разделам данных. В работе Spark запускается в локальном режиме local[*], при котором программная модель кластера сохраняется, но используются ресурсы одного компьютера.

Цель работы – освоить базовый цикл распределённой обработки данных средствами PySpark: от чтения данных и задания схемы до агрегаций, соединений, Spark SQL и записи результата в Parquet.

Задачи работы:

а) подготовить окружение и запустить локальную SparkSession;

б) загрузить таблицы базы ecommerce.sqlite в Spark DataFrame и выполнить учебную последовательность преобразований;

в) выполнить самостоятельный анализ собственного набора данных (продажи торговой компании);

г) сохранить итоговые витрины в Parquet и CSV и сформулировать выводы.

Отчёт оформлен по ГОСТ 7.32–2017 [4].

1 Подготовка окружения

Работа выполнялась на Windows 11 с Python 3.12, Java 8 и PySpark 3.5.3 (Java 8 поддерживается веткой Spark 3.5; Spark 4 требует Java 17 и выше). Код оформлен в двух Jupyter Notebook: учебный по базе ecommerce.sqlite и самостоятельный по собственным данным. Оба ноутбука выполняются последовательно командой Run All.

1.1 Установка и проверка версий

Пакеты установлены командами pip, версии Python, Java и PySpark зафиксированы в первой ячейке ноутбука (рисунок 1). Результат проверки приведён на рисунке 2.

```
import sys, subprocess, pyspark
print("Python :", sys.version.split()[0])
print("PySpark:", pyspark.__version__)
print(subprocess.run(["java", "-version"], capture_output=True, text=True).stderr.splitlines()[0])
```

Рисунок 1 – Установка пакетов и проверка версий

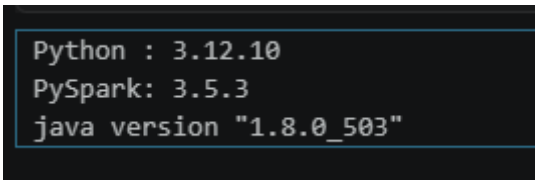


Рисунок 2 – Версии Python, PySpark и Java

1.2 Выгрузка таблиц SQLite

Spark не читает файл SQLite напрямую, поэтому использован staging-этап: модуль sqlite3 и pandas выгружают 7 нужных таблиц в CSV-файлы каталога spark_input (рисунок 3). Числа строк, которые должна напечатать программа, приведены в таблице 1; ожидаемый вывод показан на рисунке 4.

```python
from pathlib import Path
import sqlite3
import pandas as pd

DB_PATH = Path("ecommerce.sqlite")
STAGING = Path("spark_input")
STAGING.mkdir(exist_ok=True)

TABLES = [
    "orders",
    "order_items",
    "products",
    "customers",
    "order_payments",
    "order_reviews",
    "product_category_name_translation",
]

with sqlite3.connect(DB_PATH) as conn:
    for table in TABLES:
        part = pd.read_sql_query(f'SELECT * FROM "{table}"', conn)
        part.to_csv(STAGING / f"{table}.csv", index=False, encoding="utf-8")
        print(f"{table:36s} {len(part):>8} rows")
```

Рисунок 3 – Код выгрузки таблиц в staging-каталог

```
orders                                99441 rows
order_items                          112650 rows
products                              32951 rows
customers                             99441 rows
order_payments                       103886 rows
order_reviews                         97621 rows
product_category_name_translation        71 rows
```

Рисунок 4 – Результат выгрузки таблиц

Таблица 1 – Таблицы ecommerce, используемые в работе

Таблица	**Строк**	**Назначение**
orders	99 441	Заказы, статусы, даты
order_items	112 650	Позиции заказов, цена, доставка
products	32 951	Товары и их категории
customers	99 441	Клиенты и штаты
order_payments	103 886	Платежи
order_reviews	97 621	Отзывы
product_category_name_translation	71	Перевод названий категорий

2 Учебная часть: анализ базы ecommerce

2.1 Запуск SparkSession

SparkSession создана в режиме local[*] с 2 ГБ памяти драйвера и 8 разделами после shuffle (рисунок 5). В этом режиме Driver и Executor работают внутри одного процесса JVM, но модель Spark (разделы, задачи, ленивый план) остаётся такой же, как в кластере. Параметры сессии показаны на рисунке 6.

```
spark = (
    SparkSession.builder
    .appName("Practice4_Ecommerce")
    .master("local[*]")
    .config("spark.driver.memory", "2g")
    .config("spark.sql.shuffle.partitions", "8")
    .getOrCreate()
)
```

Рисунок 5 – Создание SparkSession

```
Master: local[*]
App: Practice4_Ecommerce
Spark: 3.5.3
```

Рисунок 6 – Параметры SparkSession

2.2 Чтение данных и схема

Таблица orders прочитана без вывода типов, поэтому все столбцы имеют тип string. Для order_items задана явная схема StructType (рисунок 7): числовые поля получили типы integer и double. Даты orders затем преобразованы в timestamp функцией to_timestamp. Полученные схемы приведены на рисунке 8.

```python
from pyspark.sql.types import (
    StructType, StructField, StringType, IntegerType, DoubleType
)

order_items_schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("order_item_id", IntegerType(), True),
    StructField("product_id", StringType(), True),
    StructField("seller_id", StringType(), True),
    StructField("shipping_limit_date", StringType(), True),
    StructField("price", DoubleType(), True),
    StructField("freight_value", DoubleType(), True),
])

items = (
    spark.read
    .option("header", True)
    .schema(order_items_schema)
    .csv(str(STAGING / "order_items.csv"))
)

items.printSchema()
print("Строк:", items.count())
```

Рисунок 7 – Явная схема order_items и преобразование дат

```
root
 |-- order_id: string (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- order_status: string (nullable = true)
 |-- order_purchase_timestamp: string (nullable = true)
 |-- order_approved_at: string (nullable = true)
 |-- order_delivered_carrier_date: string (nullable = true)
 |-- order_delivered_customer_date: string (nullable = true)
 |-- order_estimated_delivery_date: string (nullable = true)

Строк: 99441
+--------------------------------+--------------------------------+------------+------------------------+
|order_id                        |customer_id                     |order_status|order_purchase_timestamp|
+--------------------------------+--------------------------------+------------+------------------------+
|e481f51cbdc54678b7cc49136f2d6af7|9ef432eb6251297304e76186b10a928d|delivered   |2017-10-02 10:56:33     |
|53cdb2fc8bc7dce0b6741e2150273451|b0830fb4747a6c6d20dea0b8c802d7ef|delivered   |2018-07-24 20:41:37     |
|47770eb9100c2d0c44946d9cf07ec65d|41ce2a54c0b03bf3443c3d931a367089|delivered   |2018-08-08 08:38:49     |
|949d5b44dbf5de918fe9c16f97b45f8a|f88197465ea7920adcdbec7375364d82|delivered   |2017-11-18 19:28:06     |
|ad21c59c0840e6cb83a9ceb5573f8159|8ab97904e6daea8866dbdbc4fb7aad2c|delivered   |2018-02-13 21:18:39     |
+--------------------------------+--------------------------------+------------+------------------------+
only showing top 5 rows

root
 |-- order_id: string (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- order_status: string (nullable = true)
 |-- order_purchase_timestamp: timestamp (nullable = true)
 |-- order_approved_at: timestamp (nullable = true)
 |-- order_delivered_carrier_date: timestamp (nullable = true)
 |-- order_delivered_customer_date: timestamp (nullable = true)
 |-- order_estimated_delivery_date: timestamp (nullable = true)
```

Рисунок 8 – Схемы DataFrame после чтения

2.3 Преобразования, пропуски и агрегации

Для доставленных заказов вычислены признаки delivery_days и is_late (рисунок 9). Проверка пропусков показала, что поле order_delivered_customer_date пусто у 2 965 заказов, причём почти все они имеют статусы shipped, canceled, unavailable и другие незавершённые: пропуск отражает состояние процесса, поэтому строки не удалялись, а для категории товара использована метка unknown. Ключевые результаты приведены в таблице 2, вывод программы – на рисунке 10.

```
delivered = (
    orders
    .filter(F.col("order_status") == "delivered")
    .withColumn("purchase_month", F.date_format("order_purchase_timestamp", "yyyy-MM"))
    .withColumn(
        "delivery_days",
        F.datediff("order_delivered_customer_date", "order_purchase_timestamp")
    )
    .withColumn(
        "is_late",
        F.col("order_delivered_customer_date") > F.col("order_estimated_delivery_date")
    )
)

with_delivery_date = delivered.filter(F.col("order_delivered_customer_date").isNotNull())
late = with_delivery_date.filter(F.col("is_late"))
```

```
null_counts = orders.select([
    F.sum(F.col(c).isNull().cast("int")).alias(c) for c in timestamp_cols
])
null_counts.show(vertical=True)
```

```
missing_delivery_by_status = (
    orders
    .filter(F.col("order_delivered_customer_date").isNull())
    .groupBy("order_status")
    .count()
    .orderBy(F.desc("count"))
)
missing_delivery_by_status.show(truncate=False)
```

```
payments = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(str(STAGING / "order_payments.csv"))
)

payment_stats = (
    payments
    .groupBy("payment_type")
    .agg(
        F.count("*").alias("payments"),
        F.round(F.sum("payment_value"), 2).alias("total_value"),
        F.round(F.avg("payment_value"), 2).alias("avg_value")
    )
    .orderBy(F.desc("total_value"))
)

payment_stats.show(truncate=False)
```

Рисунок 9 – Признаки доставки, подсчёт пропусков и агрегация платежей



Рисунок 10 – Результаты преобразований, пропусков и агрегаций

Таблица 2 – Ключевые результаты учебной части

Показатель	**Значение**
Заказов delivered / с датой доставки	96 478 / 96 470
Доставлено позже ожидаемой даты	7 826 (≈ 8,1 %)
Пропусков order_delivered_customer_date	2 965
Товаров без категории	610
Позиций с price ≥ 500	3 239
Платежей credit_card	76 795 (12,54 млн)

2.4 Соединения, Spark SQL и запись результата

Позиции заказов соединены с товарами и переводом категорий двумя LEFT JOIN, число строк до и после соединения осталось 112 650, а для 1 603 позиций категория осталась unknown. Категорийная витрина построена через Spark SQL и повторена средствами DataFrame API – результаты совпали. Витрина из 74 категорий записана в Parquet и CSV (рисунок 11) и прочитана обратно (рисунок 12).

```python
translations = (
    spark.read
    .option("header", True)
    .csv(str(STAGING / "product_category_name_translation.csv"))
)

sales = (
    items.alias("i")
    .join(
        products_clean.select("product_id", "product_category_name").alias("p"),
        F.col("i.product_id") == F.col("p.product_id"),
        "left"
    )
    .join(
        translations.alias("t"),
        F.col("p.product_category_name") == F.col("t.product_category_name"),
        "left"
    )
    .select(
        F.col("i.order_id").alias("order_id"),
        F.col("i.order_item_id").alias("order_item_id"),
        F.col("i.product_id").alias("product_id"),
        F.col("i.price").alias("price"),
        F.col("i.freight_value").alias("freight_value"),
        F.coalesce(
            F.col("t.product_category_name_english"),
            F.col("p.product_category_name"),
            F.lit("unknown")
        ).alias("category")
    )
)
```

```python
sales.createOrReplaceTempView("sales")

category_sql = spark.sql("""
SELECT
    category,
    COUNT(*) AS items,
    COUNT(DISTINCT order_id) AS orders,
    ROUND(SUM(price), 2) AS revenue,
    ROUND(AVG(price), 2) AS avg_price
FROM sales
GROUP BY category
ORDER BY revenue DESC
LIMIT 10
""")

category_sql.show(truncate=False)
```

```python
OUT = Path("spark_output")
OUT.mkdir(exist_ok=True)

category_sales.write.mode("overwrite").parquet(str(OUT / "category_sales_parquet"))

(
    category_sales
    .coalesce(1)
    .write
    .mode("overwrite")
    .option("header", True)
    .csv(str(OUT / "category_sales_csv"))
)

category_back = spark.read.parquet(str(OUT / "category_sales_parquet"))

print("Строк после round-trip:", category_back.count())
category_back.printSchema()
print(sorted(p.name for p in (OUT / "category_sales_csv").iterdir()))
```

Рисунок 11 – Spark SQL и запись результата в Parquet и CSV

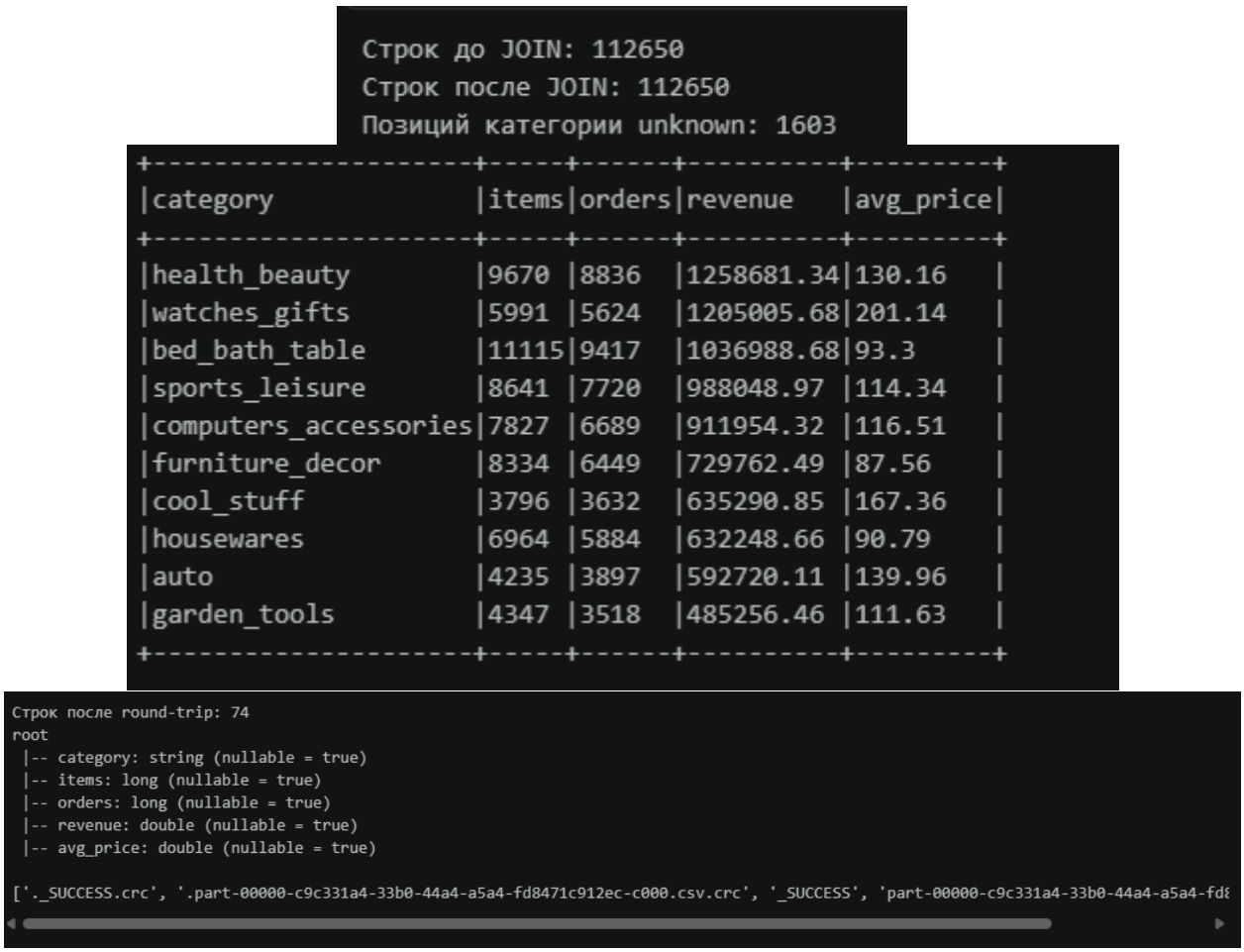


Рисунок 12 – Результаты соединений, SQL и записи

3 Самостоятельная работа

3.1 Набор данных

Для самостоятельной части выбран набор продаж торговой компании (каталог archive): факт «Продажи» и справочники «Продукты» и «Клиенты» (таблица 3). Данные обезличены: названия организаций и ИНН вымышленные, физические лица представлены единственной записью «Частное лицо». Файлы – CSV с разделителем табуляции, десятичной запятой, датами формата dd.MM.yyyy и маркером NULL для пустых значений, поэтому staging не нужен: Spark читает CSV напрямую.

Таблица 3 – Файлы собственного набора данных

Файл	**Строк**	**Содержание**
Продажи.csv	865 222	Факт: дата, филиал, товар, клиент, количество, суммы, себестоимость
Продукты.csv	21 163	Справочник товаров, брендов и групп
Клиенты.csv	3 838	Справочник клиентов (3 227 уникальных)

3.2 Схема и вычисляемые признаки

Задана явная схема с латинскими именами полей; маркер NULL превращён в null через nullValue. inferSchema с nullValue распознал числом только «Количество», а суммы и себестоимость с десятичной запятой остались строками, поэтому типы приводились явно. Добавлены признаки net_amount (сумма после скидки), margin, month и is_delivery (рисунок 13). Схема итогового DataFrame показана на рисунке 14.

```python
sales_schema = StructType([
    StructField("cost_price", StringType(), True),
    StructField("purchase_date", StringType(), True),
    StructField("receipt_no", StringType(), True),
    StructField("product_key", StringType(), True),
    StructField("branch", StringType(), True),
    StructField("quantity", StringType(), True),
    StructField("amount", StringType(), True),
    StructField("discount", StringType(), True),
    StructField("client_id", StringType(), True),
    StructField("vehicle", StringType(), True),
    StructField("address", StringType(), True),
])

sales = (
    spark.read
    .option("header", True)
    .option("sep", "\t")
    .option("nullValue", "NULL")
    .schema(sales_schema)
    .csv(str(SRC / "Продажи.csv"))
)

sales_infer = (
    spark.read.option("header", True).option("sep", "\t")
    .option("nullValue", "NULL").option("inferSchema", True)
    .csv(str(SRC / "Продажи.csv"))
```

Рисунок 13 – Приведение типов и вычисляемые признаки

```
Строк: 865222
Столбцов: 11
['Себестоимость', 'Дата покупки', 'Номер чека', 'Product_key', 'Филиал', 'Количество', 'Сумма покупки', 'Сумма скидки', 'Client_ID',
root
 |-- Себестоимость: string (nullable = true)
 |-- Дата покупки: string (nullable = true)
 |-- Номер чека: string (nullable = true)
 |-- Product_key: string (nullable = true)
 |-- Филиал: string (nullable = true)
 |-- Количество: string (nullable = true)
 |-- Сумма покупки: string (nullable = true)
 |-- Сумма скидки: string (nullable = true)
 |-- Client_ID: string (nullable = true)
 |-- Машина доставки: string (nullable = true)
 |-- Адрес: string (nullable = true)
```

```
=== inferSchema ===
root
 |-- Себестоимость: string (nullable = true)
 |-- Дата покупки: string (nullable = true)
 |-- Номер чека: string (nullable = true)
 |-- Product_key: string (nullable = true)
 |-- Филиал: string (nullable = true)
 |-- Количество: integer (nullable = true)
 |-- Сумма покупки: string (nullable = true)
 |-- Сумма скидки: string (nullable = true)
 |-- Client_ID: string (nullable = true)
 |-- Машина доставки: string (nullable = true)
 |-- Адрес: string (nullable = true)

=== явная схема ===
root
 |-- cost_price: string (nullable = true)
 |-- purchase_date: string (nullable = true)
 |-- receipt_no: string (nullable = true)
 |-- product_key: string (nullable = true)
 |-- branch: string (nullable = true)
 |-- quantity: string (nullable = true)
 |-- amount: string (nullable = true)
 |-- discount: string (nullable = true)
 |-- client_id: string (nullable = true)
 |-- vehicle: string (nullable = true)
 |-- address: string (nullable = true)

Строк: 865222
```

Рисунок 14 – Схемы собственного набора данных

3.3 Пропуски

Пропуски найдены в четырёх полях. Решения по каждому из них и их обоснование приведены в таблице 4. Строки с null не удалялись: пропуски объясняются процессом продаж.

Таблица 4 – Пропуски и принятые решения

Поле	**Пропусков**	**Причина и решение**
address	832 181	Самовывоз, доставки нет; строки оставлены, создан признак is_delivery
cost_price	86 431	Себестоимость неизвестна; нулём не заполнялась, маржа считается по известным
receipt_no	3 934	Отгрузки без чека; заполнено меткой NO_RECEIPT
quantity	100	Количество не указано; не заполнялось, сумма продажи известна

3.4 Агрегации, соединения и Spark SQL

Показатели по филиалам вычислены семью агрегатами. Справочник клиентов содержал 566 дублирующихся ключей, поэтому перед соединением он был дедуплицирован: иначе JOIN размножил бы факты. После LEFT JOIN с товарами и клиентами и INNER JOIN с клиентами число строк осталось 865 222. Построено три запроса Spark SQL, один из них с GROUP BY и сортировкой по агрегату (рисунок 15). Результаты по филиалам – в таблице 5, вывод запросов – на рисунке 16.

```python
branch_stats = (
    sales_c
    .groupBy("branch")
    .agg(
        F.count("*").alias("lines"),
        F.countDistinct("receipt_no").alias("receipts"),
        F.countDistinct("client_id").alias("clients"),
        F.round(F.sum("amount"), 0).alias("gross"),
        F.round(F.sum("discount"), 0).alias("discounts"),
        F.round(F.sum("net_amount"), 0).alias("net_revenue"),
        F.round(F.avg("discount_pct"), 2).alias("avg_discount_pct"),
    )
    .orderBy(F.desc("net_revenue"))
)
branch_stats.show(truncate=False)
```

```python
clients_dupes = (clients_raw.groupBy("client_id").count().filter("count > 1"))
print("Клиентов с дублями:", clients_dupes.count())

clients = (
    clients_raw
    .groupBy("client_id")
    .agg(F.min("client_name").alias("client_name"))
)
print("clients после дедупликации:", clients.count())

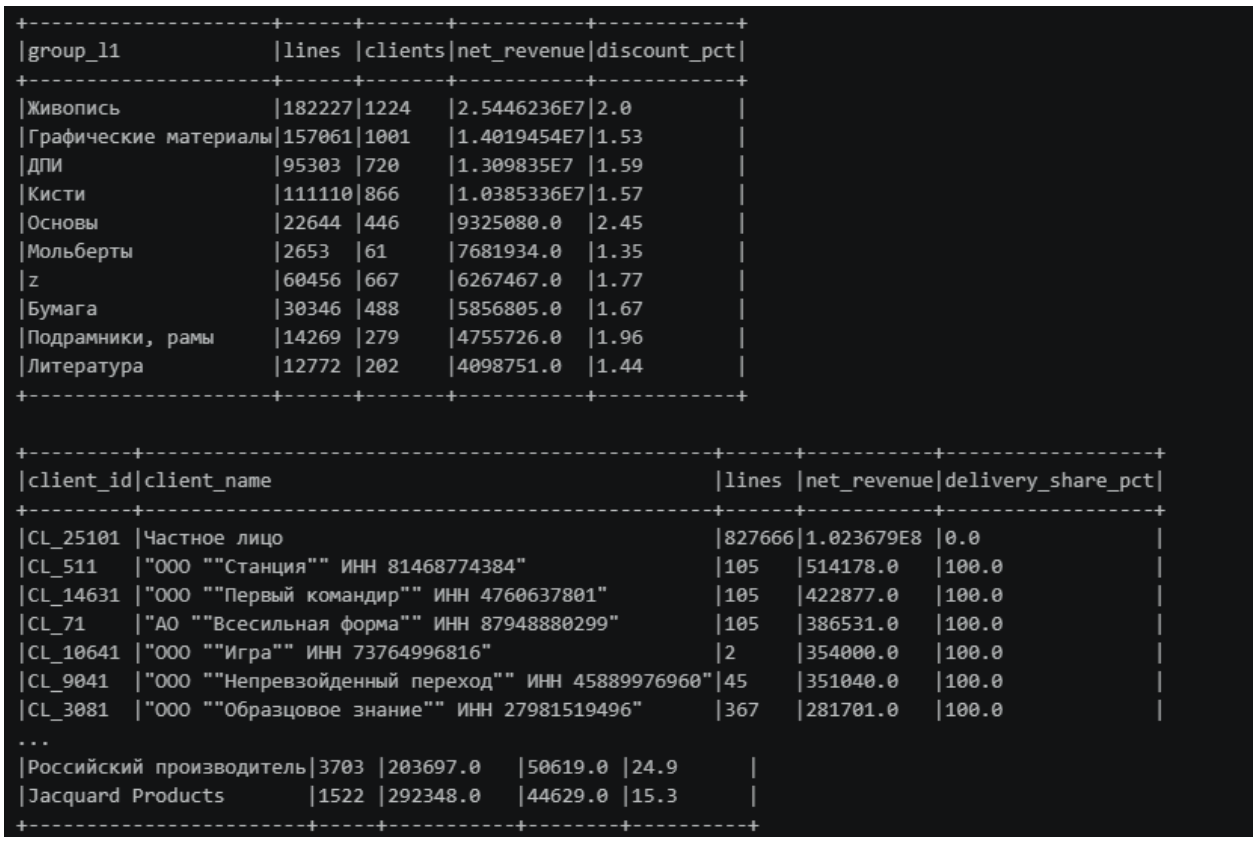
n_before = sales_c.count()

sales_p = (
    sales_c.alias("s")
    .join(products.alias("p"), F.col("s.product_key") == F.col("p.product_key"), "left")
    .select("s.*", "p.product_name", "p.brand", "p.product_group")
    .withColumn("brand", F.coalesce("brand", F.lit("unknown")))
    .withColumn("group_l1", F.split("product_group", "-").getItem(0))
)
print("LEFT JOIN с товарами  до:", n_before, " после:", sales_p.count())
print("Строк без товара в справочнике:", sales_p.filter(F.col("product_name").isNull()).count())
```

Рисунок 15 – JOIN со справочниками и запрос Spark SQL

```
+--------------------------+------+--------+-------+-----------+---------+-----------+----------------+
|branch                    |lines |receipts|clients|gross      |discounts|net_revenue|avg_discount_pct|
+--------------------------+------+--------+-------+-----------+---------+-----------+----------------+
|ТЗ-2 ( Ленинградское шоссе)|431372|78785   |406    |5.8750803E7|634368.0 |5.8116436E7|1.02            |
|ТЗ-1 ( ул.Фадеева)        |238046|46338   |1469   |4.4343168E7|1114995.0|4.3228173E7|1.35            |
|ТЗ-3 (Курский)            |190730|32297   |303    |2.6249056E7|498665.0 |2.5750391E7|1.33            |
|Багетная мастерская       |848   |1       |1      |1399096.0  |0.0      |1399096.0  |0.0             |
|РС (Распределительный склад)|249   |1       |9      |788680.0   |27859.0  |760821.0   |3.61            |
|Кафе-1 ( ул.Фадеева)      |3977  |2459    |2      |207762.0   |672.0    |207091.0   |0.33            |
+--------------------------+------+--------+-------+-----------+---------+-----------+----------------+
```

```
Клиентов с дублями: 566
clients после дедупликации: 3227
LEFT JOIN с товарами  до: 865222  после: 865222
Строк без товара в справочнике: 0
```



```
+--------------------+------+-------+-----------+------------+
|group_l1            |lines |clients|net_revenue|discount_pct|
+--------------------+------+-------+-----------+------------+
|Живопись            |182227|1224   |2.5446236E7|2.0         |
|Графические материалы|157061|1001   |1.4019454E7|1.53        |
|ДПИ                 |95303 |720    |1.309835E7 |1.59        |
|Кисти               |111110|866    |1.0385336E7|1.57        |
|Основы              |22644 |446    |9325080.0  |2.45        |
|Мольберты           |2653  |61     |7681934.0  |1.35        |
|z                   |60456 |667    |6267467.0  |1.77        |
|Бумага              |30346 |488    |5856805.0  |1.67        |
|Подрамники, рамы    |14269 |279    |4755726.0  |1.96        |
|Литература          |12772 |202    |4098751.0  |1.44        |
+--------------------+------+-------+-----------+------------+


+---------+-------------------------------------------------+------+-----------+------------------+
|client_id|client_name                                      |lines |net_revenue|delivery_share_pct|
+---------+-------------------------------------------------+------+-----------+------------------+
|CL_25101 |Частное лицо                                     |827666|1.023679E8 |0.0               |
|CL_511   |"ООО ""Станция"" ИНН 81468774384"                |105   |514178.0   |100.0             |
|CL_14631 |"ООО ""Первый командир"" ИНН 4760637801"         |105   |422877.0   |100.0             |
|CL_71    |"АО ""Всесильная форма"" ИНН 87948880299"        |105   |386531.0   |100.0             |
|CL_10641 |"ООО ""Игра"" ИНН 73764996816"                   |2     |354000.0   |100.0             |
|CL_9041  |"ООО ""Непревзойденный переход"" ИНН 45889976960"|45    |351040.0   |100.0             |
|CL_3081  |"ООО ""Образцовое знание"" ИНН 27981519496"      |367   |281701.0   |100.0             |
...
|Российский производитель|3703 |203697.0   |50619.0 |24.9      |
|Jacquard Products       |1522 |292348.0   |44629.0 |15.3      |
+------------------------+-----+-----------+--------+----------+
```

Рисунок 16 – Результаты агрегаций, соединений и запросов

Таблица 5 – Показатели по филиалам (чистая выручка после скидок)

Филиал	**Строк**	**Выручка, млн**	**Средняя скидка, %**
ТЗ-2 (Ленинградское шоссе)	431 372	58,12	1,02
ТЗ-1 (ул. Фадеева)	238 046	43,23	1,35
ТЗ-3 (Курский)	190 730	25,75	1,33
Остальные (3 подразделения)	5 074	2,37	0–3,61

3.5 Экспорт результата

Итоговая витрина «месяц × филиал × группа товаров» (1 780 строк) записана в Parquet, агрегат по филиалам – дополнительно в CSV. После повторного чтения количество строк (1 780), имена и типы столбцов и сумма net_revenue (129,46 млн) совпали с исходными (рисунок 17).

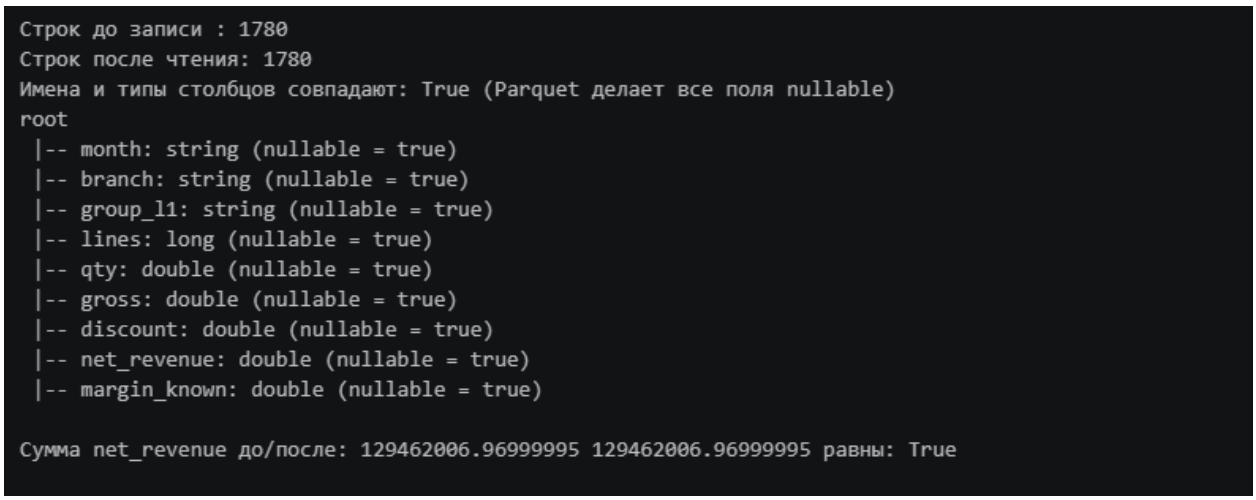


Рисунок 17 – Результат повторного чтения Parquet

ЗАКЛЮЧЕНИЕ

В ходе работы освоен базовый цикл распределённой обработки данных в Apache Spark, все задачи выполнены. Запущена SparkSession в режиме local[*], таблицы ecommerce.sqlite прочитаны в DataFrame с явной и неявной схемами, выполнены преобразования, обработка пропусков, агрегации, соединения и запросы Spark SQL.

В учебной части показано, что пропуск даты доставки связан со статусом заказа, поэтому удалять такие строки нельзя, а около 8,1 % доставленных заказов пришли позже ожидаемой даты. LEFT JOIN сохранил все 112 650 позиций заказов.

В самостоятельной части проанализированы продажи торговой компании (865 тыс. строк): почти 96 % строк и около 102 млн из 129 млн выручки приходится на розничного покупателя, основную выручку дают три торговых зала, наиболее крупная товарная группа – «Живопись». Ограничения анализа: расчёт маржи ориентировочный (у 13 % строк с известной себестоимостью маржа отрицательна, а у 10 % строк себестоимость неизвестна), а данные охватывают неполный период с июня 2020 года.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Apache Spark Documentation / The Apache Software Foundation. – URL: https://spark.apache.org/docs/3.5.3/ (дата обращения: 19.09.2026).

2. PySpark API Reference / The Apache Software Foundation. – URL: https://spark.apache.org/docs/3.5.3/api/python/ (дата обращения: 19.09.2026).

3. Практическая работа № 3. Распределённая модель обработки данных: методические материалы. – РТУ МИРЭА, 2026.

4. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – М.: Стандартинформ, 2017.

5. pandas Documentation / pandas via NumFOCUS. – URL: https://pandas.pydata.org/docs/ (дата обращения: 19.09.2026).